

APOLLO INSTITUTE OF COMPUTER APPLICATION

BACHELOR OF COMPUTER APPLICATION

SEMESTER 03

Object Oriented Programming with Java (BC03001021)

Lecture Notes

UNIT 4. Inheritance, Packages & Interfaces (12 HOURS)

Lecture	Topics Covered
1	Introduction to Inheritance, Superclass, Subclass
2	Types of Inheritance – Single & Multilevel
3	Hierarchical, Multiple & Hybrid Inheritance
4	Method Overriding & super
5	Dynamic Method Dispatch
6	Object Class
7	Packages – Creating & Importing
8	Package Access Rules & Class Hiding
9	Interfaces – Definition & Implementation
10	Interface Inheritance & Multiple Inheritance
11	Abstract Class
12	Final Class + Complete Revision & Comparison

LECTURE 1 – Introduction to Inheritance

1. What is Inheritance?

Inheritance is an OOP mechanism through which one class acquires the properties and behaviors of another class.

It promotes:

- Code reusability
- Extensibility
- Method overriding
- Hierarchical classification

Basic terminology

Superclass / Parent class:

The class whose properties are inherited.

Subclass / Child class:

The class that inherits properties from another class.

Syntax

```
class Parent {  
    int x = 10;  
  
    void display() {  
        System.out.println("Parent class");  
    }  
}
```

```
class Child extends Parent {  
    void show() {  
        System.out.println("Child class");  
    }  
}
```

Here:

- Parent → Superclass
- Child → Subclass
- extends → keyword used for inheritance

Example

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Demo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output:

Animal eats
Dog barks

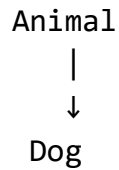
Advantages

1. Code reusability
2. Easy maintenance
3. Supports method overriding
4. Reduces code duplication
5. Helps implement polymorphism

LECTURE 2 – Types of Inheritance: Single & Multilevel

1. Single Inheritance

When one subclass inherits from one superclass, it is called **single inheritance**.

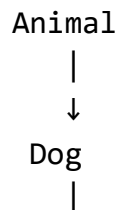


Example

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}
```

2. Multilevel Inheritance

When a class inherits from another derived class, it is called **multilevel inheritance**.



↓
Puppy

Example

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}  
  
class Puppy extends Dog {  
    void play() {  
        System.out.println("Playing");  
    }  
}
```

Puppy can access methods of both Dog and Animal.

LECTURE 3 – Hierarchical, Multiple & Hybrid Inheritance

1. Hierarchical Inheritance

When multiple subclasses inherit from a single superclass.

```
Animal  
/    \  

```



Example

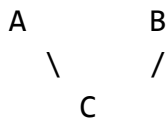
```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Meowing");
    }
}
```

2. Multiple Inheritance

When one class inherits from **more than one class**, it is called multiple inheritance.



Important point in Java

Java **does not support multiple inheritance through classes**.

For example, this is not allowed:

```
class A { }
class B { }
```

```
// Not allowed  
class C extends A, B { }
```

Why?

It can create **ambiguity**.

Suppose both A and B have a method called show():

```
A → show()  
B → show()  
  \  /  
   C
```

If C calls show(), Java cannot determine which show() should be used.

This is called the **Diamond Problem / Ambiguity**.

Java solves this using **interfaces**.

3. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

Java does not support hybrid inheritance through classes if it results in multiple inheritance.

Interfaces can be used to achieve similar designs.

LECTURE 4 – Method Overriding & super

Method Overriding

When a subclass provides its own implementation of a method already defined in its superclass, it is called **method overriding**.

Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Here, Dog overrides sound().

Rules

1. Method must have the same name.
2. Parameter list must be the same.
3. Return type must be same or compatible.
4. The overridden method cannot have more restrictive access.
5. final methods cannot be overridden.
6. static methods are hidden, not overridden in the normal polymorphic sense.

super Keyword

super refers to the immediate parent class object/member.

It can be used to:

1. Access superclass variable
2. Call superclass method
3. Call superclass constructor

Access superclass variable

```
class A {
    int x = 10;
}

class B extends A {
    int x = 20;

    void display() {
        System.out.println(x);
        System.out.println(super.x);
    }
}
```

Output:

```
20
10
```

Calling superclass method

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        super.show();
        System.out.println("B");
    }
}
```

LECTURE 5 – Dynamic Method Dispatch

What is Dynamic Method Dispatch?

Dynamic method dispatch is the mechanism by which an overridden method is called based on the **actual object at runtime**, rather than the reference type.

It is an important feature of **runtime polymorphism**.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}

class Demo {
    public static void main(String[] args) {

        Animal a;

        a = new Dog();
        a.sound();
    }
}
```

```
        a = new Cat();
        a.sound();
    }
}
```

Output

Dog barks
Cat meows

Important concept

```
Animal a = new Dog();
```

- Reference type → Animal
- Object type → Dog

At runtime, Java selects the Dog version of sound().

Exam definition

Dynamic method dispatch is the process by which a call to an overridden method is resolved at runtime according to the actual object that the reference variable refers to.

LECTURE 6 – Object Class

Every Java class directly or indirectly inherits from the **Object class**.

Therefore:

```
Object
|
↓
Animal
|
↓
Dog
```

The Object class is the root class of the Java class hierarchy.

Important methods of Object class

Method	Purpose
toString()	Returns string representation of object
equals()	Compares objects
hashCode()	Returns hash code
getClass()	Returns runtime class information
clone()	Creates a copy where supported
finalize()	Historical/deprecated mechanism; should not be used in modern Java

Example: toString()

```
class Student {
    int roll = 101;

    public String toString() {
        return "Roll = " + roll;
    }
}

class Demo {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s);
    }
}
```

Output:

Roll = 101

Important point

If a class does not explicitly extend another class:

```
class Student {
}
```

it still ultimately inherits from Object.

LECTURE 7 – Packages: Creating & Importing

What is a Package?

A **package** is a collection of related classes, interfaces and other Java elements.

Packages are used for:

- Organizing classes
- Avoiding naming conflicts
- Providing access protection
- Code organization

Creating a Package

```
package mypack;
```

```
public class Student {  
    public void display() {  
        System.out.println("Hello Student");  
    }  
}
```

Save the file as:

Student.java

Compile:

```
javac -d . Student.java
```

This creates:

```
mypack
└─ Student.class
```

Importing a Package

Import a particular class

```
import mypack.Student;
```

Import all classes

```
import mypack.*;
```

Example

```
import mypack.Student;

class Demo {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Fully Qualified Name

We can also use a class without importing it:

```
mypack.Student s = new mypack.Student();
```

LECTURE 8 – Package Access Rules & Class Hiding

Access Modifiers

Java provides four levels of access:

Modifier	Same Class	Same Package	Subclass outside Package	Other Package
private	✓	X	X	X
Default	✓	✓	X	X
protected	✓	✓	✓*	X*
public	✓	✓	✓	✓

protected access outside the package is available through inheritance, subject to Java's access rules.

Default Access

If no access modifier is specified:

```
class Student {  
    int rollNo;  
}
```

The member has **package-private/default access**.

It can be accessed by classes in the same package.

Class Hiding / Name Conflict

Suppose two packages contain classes with the same name:

```
package p1 → Student  
package p2 → Student
```

If both are imported using wildcard imports:

```
import p1.*;  
import p2.*;
```

and we write:

```
Student s;
```

Java may report ambiguity because it cannot determine which Student is intended.

Solution

Use the fully qualified class name:

```
p1.Student s1;  
p2.Student s2;
```

Important distinction

Method overriding and **class-name ambiguity** are different concepts.

LECTURE 9 – Interfaces

What is an Interface?

An **interface** is a reference type used to define a contract that implementing classes must follow.

An interface is declared using the `interface` keyword.

Syntax

```
interface Animal {  
    void sound();  
}
```

A class implements an interface using implements.

```
class Dog implements Animal {  
  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Complete example

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {  
  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
class Demo {  
    public static void main(String[] args) {  
        Circle c = new Circle();  
        c.draw();  
    }  
}
```

Important Interface Points

Traditionally, interface methods were implicitly:

```
public abstract
```

and fields were:

```
public static final
```

Modern Java interfaces can also contain:

- default methods
- static methods
- private methods

Example

```
interface Vehicle {  
    void start();  
}
```

The implementing class must provide the required implementation.

LECTURE 10 – Interface Inheritance & Multiple Inheritance

Interface Inheritance

One interface can inherit another interface using extends.

```
interface A {  
    void methodA();  
}
```

```
interface B extends A {  
    void methodB();  
}
```

A class implementing B must implement both methods.

```

class Demo implements B {

    public void methodA() {
        System.out.println("Method A");
    }

    public void methodB() {
        System.out.println("Method B");
    }
}

```

Multiple Inheritance Using Interfaces

Java can achieve multiple inheritance through interfaces.

```

interface A {
    void showA();
}

interface B {
    void showB();
}

class C implements A, B {

    public void showA() {
        System.out.println("A");
    }

    public void showB() {
        System.out.println("B");
    }
}

```

Here:

```

A      B
 \    /

```



C implements both A and B.

Why interfaces?

Java avoids multiple inheritance of classes because of ambiguity, while interfaces provide a controlled mechanism for implementing multiple contracts.

Interface Extending Multiple Interfaces

An interface itself can extend multiple interfaces:

```
interface A {  
    void showA();  
}
```

```
interface B {  
    void showB();  
}
```

```
interface C extends A, B {  
    void showC();  
}
```

LECTURE 11 – Abstract Class

What is an Abstract Class?

A class declared with the `abstract` keyword is called an **abstract class**.

It may contain:

- Abstract methods

- Concrete methods
- Variables
- Constructors
- Static members

Syntax

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

An abstract class **cannot be instantiated directly**.

This is invalid:

```
Animal a = new Animal(); // Error
```

Abstract Method

A method declared without a body is called an abstract method.

```
abstract void sound();
```

A concrete subclass must implement it unless the subclass is also abstract.

Example

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```

}

class Dog extends Animal {

    void sound() {
        System.out.println("Dog barks");
    }
}

class Demo {
    public static void main(String[] args) {

        Dog d = new Dog();

        d.sound();
        d.eat();
    }
}

```

Output

```

Dog barks
Animal eats

```

LECTURE 12 – Final Class + Complete Revision

1. Final Class

A class declared using the `final` keyword cannot be inherited.

Syntax

```

final class Vehicle {
    void display() {
        System.out.println("Vehicle");
    }
}

```

```
    }  
}
```

We cannot write:

```
class Car extends Vehicle {  
}
```

because Vehicle is a final class.

Example

```
final class Student {  
  
    void display() {  
        System.out.println("Student");  
    }  
}  
  
class Demo {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

Why use a final class?

A final class is used when we want to prevent inheritance.

Examples include Java classes such as:

String

which is a final class.

Final Keyword – Three Uses

The final keyword can be applied to a:

1. Variable

```
final int x = 10;
```

Its value cannot be reassigned.

```
x = 20; // Error
```

2. Method

```
class A {  
    final void show() {  
        System.out.println("Hello");  
    }  
}
```

The method cannot be overridden.

3. Class

```
final class A {  
}
```

The class cannot be extended.

Important Comparison: Abstract Class vs Interface

Abstract Class

Declared using abstract class

Interface

Declared using interface

Can contain abstract and concrete methods

Can have constructors

Can have instance variables

A class extends an abstract class

A class can extend only one class

Used for common base implementation/state

Can define abstract methods and, in modern Java, default/static/private methods

Cannot have constructors

Fields are implicitly `public static final`

A class implements an interface

A class can implement multiple interfaces

Used mainly to define a contract/capability

Important Comparison: extends vs implements

extends

Used for class inheritance:

```
class Dog extends Animal {  
}
```

Also used when one interface inherits another:

```
interface B extends A {  
}
```

implements

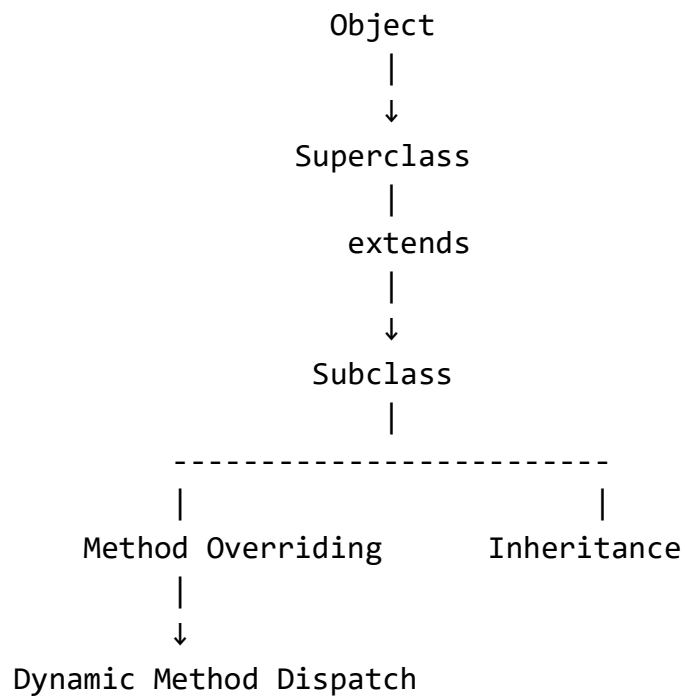
Used when a class implements an interface:

```
class Dog implements Animal {  
}
```

A class can implement multiple interfaces:

```
class Demo implements A, B {  
}
```

Complete Unit 4 Concept Map



Packages

- Create package
- Import package
- Access rules

Interfaces

- Define interface
- Implement interface
- Interface inheritance
- Multiple inheritance

Classes

|

- └ Abstract Class
- └ Final Class

Key GTU Exam Points

Students should be able to explain:

1. Define inheritance with an example.
2. Explain different types of inheritance.
3. Why does Java not support multiple inheritance through classes?
4. Explain ambiguity/diamond problem.
5. Explain method overriding.
6. Explain the use of super.
7. Explain dynamic method dispatch with example.
8. Explain the Object class.
9. Define package and explain how to create/import it.
10. Explain access rules in packages.
11. Define interface with example.
12. Explain multiple inheritance using interfaces.
13. Explain abstract class with example.
14. Explain final class.
15. Differentiate **abstract class and interface**.
16. Differentiate **method overriding and method overloading**.
17. Explain the uses of the final keyword.
18. Explain extends and implements.

Suggested practical programs for Lecture 12 revision

1. Single inheritance program
2. Multilevel inheritance program
3. Method overriding program
4. Dynamic method dispatch program
5. Package creation and importing program
6. Interface implementation program
7. Multiple inheritance using interfaces
8. Abstract class program
9. Final class program
10. Program demonstrating Object class methods

